The Interactions API (/gemini-api/docs/interactions-overview) is now generally available. We recommend using this API for access to all the latest features and models.

Getting started

This guide gets you started with the Gemini API using the Interactions API (/gemini-api/docs/interactions-overview). You'll make your first API call in under a minute and explore text generation, multimodal understanding, image generation, structured output, tools, function calling, agents, and background execution.

Using a coding agent? Install the skill so your agent stays current with Interactions API patterns:

```
npx skills add google-gemini/gemini-skills --skill gemini-interactions-api
```

The Interactions API is available through the Python (https://github.com/googleapis/python-genai) and JavaScript (https://github.com/googleapis/js-genai) SDKs, as well as through REST.

1. Get an API key

To use the Gemini API, you need to have an API key to authenticate your requests, enforce security limits, and track usage to your account.

- Google AI Studio automatically creates a project and API key for new users. You can copy it from the API keys page (https://aistudio.google.com/api-keys).
- If you need a new key, click **Create API key** in AI Studio and follow the dialog to add a new key-project pair.

Create a Gemini API Key (https://aistudio.google.com/apikey)

Set your key as an environment variable:

```
export GEMINI_API_KEY="YOUR_API_KEY"
```

Upgrade to the paid tier

Upgrading to the paid tier increases your rate limits and requires setting up Cloud Billing.

- Click **Set up billing** on the AI Studio API keys (https://aistudio.google.com/api-keys) or Projects (https://aistudio.google.com/projects) pages.
- Follow the Cloud Billing dialog to create or link a billing account, add a payment method, and prepay a minimum of $10 (or currency equivalent) in paid credits.

- View your API usage in Google AI Studio (https://aistudio.google.com/usage) under **Dashboard** > **Usage**.

See the Billing page (/gemini-api/docs/billing) for more information.

2. Install the SDK and make your first call

Install the SDK and generate text with a single API call.

PythonJavaScript (#javascript)REST (#rest)
(#python)

Install the SDK:

```
pip install -U google-genai
```

Initialize the client and make a request:

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.6-flash",
    input="Explain how AI works in a few words"
)
print(interaction.output_text)
```

Response:


```
{
  "id": "v1_ChdpQUFvYXI...",
  "status": "completed",
  "usage": {
    "total_tokens": 197,
    "total_input_tokens": 8,
    "total_output_tokens": 12
  },
  "created": "2026-06-09T12:01:25Z",
  "steps": [
    {
      "type": "thought",
      "signature": "EvEFCu4FAQw..."
    },
    {
      "type": "model_output",
      "content": [
        {
          "type": "text",
          "text": "AI learns patterns from data, then uses those patterns to make pre
```

```
        }
      ]
    }
  ],
  "object": "interaction",
  "model": "gemini-3.6-flash",
}
```

When using REST, the API returns the full `Interaction` resource containing metadata, usage statistics, and the step-by-step history of the turn.

While the SDKs expose the full response, they also provide convenience properties like `interaction.output_text` and `interaction.output_image` to access final outputs directly. Learn more about the response structure in the Interactions overview (/gemini-api/docs/interactions-overview) or read the text generation guide (/gemini-api/docs/text-generation) for details on system instructions and generation config.

3. Stream the response

For more fluid interactions, stream the response as it's generated. Each `step.delta` event delivers a chunk of text you can display immediately.

Python JavaScript (#javascript) REST (#rest)

(#python)

```python
from google import genai

client = genai.Client()

stream = client.interactions.create(
    model="gemini-3.6-flash",
    input="Explain how AI works",
    stream=True
)
for event in stream:
    print(event)
```

When streaming, the server responds with a stream of server-sent events (SSE). Each event includes a type and JSON data.

Response:

```
event: interaction.created
data: {"interaction":{"id":"v1_Chd...","status":"in_progress","model":"gemini-3.6-fla

event: step.start
data: {"index":0,"step":{"type":"thought"},"event_type":"step.start"}

event: step.delta
data: {"index":0,"delta":{"signature":"EvEFCu4F...","type":"thought_signature"},"ever

event: step.stop
```

```
data: {"index":0,"event_type":"step.stop"}

event: step.start
data: {"index":1,"step":{"type":"model_output"},"event_type":"step.start"}

event: step.delta
data: {"index":1,"delta":{"text":"AI ","type":"text"},"event_type":"step.delta"}

event: step.delta
data: {"index":1,"delta":{"text":"works ","type":"text"},"event_type":"step.delta"}

event: step.stop
data: {"index":1,"event_type":"step.stop"}

event: interaction.completed
data: {"interaction":{"id":"v1_Chd...","status":"completed","usage":{"total_tokens":
```

For a detailed look at handling streaming events and delta types, see the streaming interactions guide (/gemini-api/docs/streaming).

4. Multi-turn conversations

The Interactions API supports multi-turn conversations with two approaches:

- **Stateful (recommended)**: Continue a conversation on the server using `previous_interaction_id`. Ideal for most chat and agentic workflows where you want the server to manage history and optimize caching.
- **Stateless**: Manage the conversation history on the client by passing all previous turns (including intermediate model thought and tool steps) in each request.

Stateful (recommended) Stateless (#stateless)
(#stateful-recommended)

Chain interactions by passing `previous_interaction_id`. The server manages the full conversation history for you.

Python JavaScript (#javascript) REST (#rest)
(#python)

```
from google import genai

client = genai.Client()

# Server-side state (recommended)
interaction1 = client.interactions.create(
    model="gemini-3.6-flash",
    input="I have 2 dogs in my house.",
)
print("Response 1:", interaction1.output_text)

interaction2 = client.interactions.create(
    model="gemini-3.6-flash",
    input="How many paws are in my house?",
    previous_interaction_id=interaction1.id,
```

```
)
print("Response 2:", interaction2.output_text)
```

Response:


```
{
  "id": "v2_Chd...",
  "status": "completed",
  "usage": {
    "total_tokens": 240,
    "total_input_tokens": 60,
    "total_output_tokens": 20
  },
  "steps": [
    {
      "type": "model_output",
      "content": [
        {
          "type": "text",
          "text": "There are 8 paws in your house. 2 dogs \u00d7 4 paws = 8 paws."
        }
      ]
    }
  ],
  "object": "interaction",
  "model": "gemini-3.6-flash"
}
```


The second interaction returns a complete response object that includes only the new steps, but is grounded in the previous turn's context. Learn more about maintaining state in the multi-turn conversations guide (/gemini-api/docs/text-generation#multi-turn-conversations), or explore stateless mode (/gemini-api/docs/text-generation#stateless-conversations) for client-side history management.

5. Multimodal understanding

Gemini models understand images, audio, video, and documents natively. Pass media alongside text in a single request.

Python JavaScript (#javascript) REST (#rest)
(#python)

```
import base64
from google import genai

client = genai.Client()

# Load a local image
with open("sample.jpg", "rb") as f:
    image_bytes = f.read()
image_b64 = base64.b64encode(image_bytes).decode("utf-8")

interaction = client.interactions.create(
```

```
    model="gemini-3.6-flash",
    input=[
        {"type": "text", "text": "Compare this local image and this remote audi
        {
            "type": "image",
            "data": image_b64,
            "mime_type": "image/jpeg"
        },
        {
            "type": "audio",
            "uri": "https://storage.googleapis.com/generativeai-downloads/data/
            "mime_type": "audio/mp3"
        }
    ]
)
print(interaction.output_text)
```

Response:


```
{
  "id": "v1_Chd...",
  "status": "completed",
  "usage": {
    "total_tokens": 300
  },
  "steps": [
    {
      "type": "model_output",
      "content": [
        {
          "type": "text",
          "text": "The local image displays a pipe organ while the remote audio file
        }
      ]
    }
  ],
  "object": "interaction",
  "model": "gemini-3.6-flash",
}
```


Explore how to pass images, video, and audio files in the image understanding guide (/gemini-api/docs/image-understanding).

Audio understanding (/gemini-api/docs/audio)

Transcribe, summarize, or answer questions about audio files.

Video understanding (/gemini-api/docs/video-understanding)

Analyze video content, locate events, and describe actions.

Document processing

Extract information from PDFs and other document formats.

(/gemini-api/docs/document-processing)

6. Multimodal generation

Gemini can generate images natively using the Nano Banana (/gemini-api/docs/image-generation) image models.

Python JavaScript (#javascript) REST (#rest)
(#python)

```
import base64
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.1-flash-image",
    input="Generate an image of a futuristic city skyline at sunset",
)

with open("generated_image.png", "wb") as f:
    f.write(base64.b64decode(interaction.output_image.data))
```

Response:

```
{
  "id": "v1_Chd...",
  "status": "completed",
  "steps": [
    {
      "type": "model_output",
      "content": [
        {
          "type": "image",
          "data": "BASE64_ENCODED_IMAGE",
          "mime_type": "image/png"
        }
      ]
    }
  ],
  "object": "interaction",
  "model": "gemini-3.1-flash-image",
}
```

When the model generates an image, it returns the base64-encoded image data in a step within the `steps` array, as well as via the `output_image` convenience property. Check out the image generation guide (/gemini-api/docs/image-generation) to learn about aspect ratios, image editing, and references.

Speech generation (/gemini-api/docs/speech-generation)

Generate expressive, multi-speaker speech with Gemini 3.1 Flash TTS.

Music generation (/gemini-api/docs/music-generation)

Create clips and full-length songs with Lyria 3.

7. Use structured output

Configure the model to return JSON that matches a schema you define. Structured output works with Pydantic (https://docs.pydantic.dev/latest/) (Python) and Zod (https://zod.dev/) (JavaScript).

Python JavaScript (#javascript) REST (#rest)
(#python)

```
from google import genai
from pydantic import BaseModel, Field
from typing import List, Optional

class Recipe(BaseModel):
    recipe_name: str = Field(description="Name of the recipe.")
    ingredients: List[str] = Field(description="List of ingredients.")
    prep_time_minutes: Optional[int] = Field(description="Prep time in minutes.

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.6-flash",
    input="Give me a recipe for banana bread",
    response_format={
        "type": "text",
        "mime_type": "application/json",
        "schema": Recipe.model_json_schema()
    },
)

recipe = Recipe.model_validate_json(interaction.output_text)
print(recipe)
```

Response:

```
{
  "id": "v1_Chd...",
  "status": "completed",
  "steps": [
    {
      "type": "model_output",
      "content": [
        {
```

```
          "type": "text",
          "text": "{\n  \"recipe_name\": \"Classic Banana Bread\",\n  \"ingredients\"
        }
      ]
    }
  ],
  "object": "interaction",
  "model": "gemini-3.6-flash",
}
```

The output text block contains a valid JSON string conforming exactly to the requested schema. To learn how to define more complex structures and recursive schemas, see the structured output guide (/gemini-api/docs/structured-output).

8. Use tools

Ground the model's response in real-time information with Google Search. The API automatically searches, processes results, and returns citations.

Python JavaScript (#javascript) REST (#rest)
(#python)

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.6-flash",
    input="Who won the euro 2024?",
    tools=[{"type": "google_search"}]
)

print(interaction.output_text)

# Print citations
for step in interaction.steps:
    if step.type == "model_output":
        for content_block in step.content:
            if content_block.type == "text" and content_block.annotations:
                print("\nCitations:")
                for annotation in content_block.annotations:
                    if annotation.type == "url_citation":
                        print(f"  [{annotation.title}]({annotation.url})")
```

Response:

```
{
  "id": "v1_Chd...",
  "status": "completed",
  "steps": [
    {
      "type": "thought",
```

```
      "signature": "EvEFCu4F..."
    },
    {
      "type": "google_search_call",
      "arguments": {
        "queries": ["UEFA Euro 2024 winner"]
      }
    },
    {
      "type": "google_search_result",
      "call_id": "search_001",
      "result": [
        {
          "search_suggestions": "<!-- HTML and CSS search widget -->"
        }
      ]
    },
    {
      "type": "model_output",
      "content": [
        {
          "type": "text",
          "text": "Spain won Euro 2024, defeating England 2-1 in the final.",
          "annotations": [
            {
              "type": "url_citation",
              "url": "https://www.uefa.com/euro2024",
              "title": "uefa.com",
              "start_index": 0,
              "end_index": 56
            }
          ]
        }
      ]
    }
  ],
  "object": "interaction",
  "model": "gemini-3.6-flash",
}
```


The search steps are detailed within the interaction history, and the final output includes inline citations pointing to web sources.

You can learn how to extract search citations in the Google Search grounding guide (/gemini-api/docs/google-search), or see how to combine multiple tools in the tool combination guide (/gemini-api/docs/tool-combination).



Code execution (/gemini-api/docs/code-execution)

Run Python code in a secure sandboxed Borg environment.



URL context (/gemini-api/docs/url-context)

Pass public web URLs directly to ground responses in webpage content.

File search (/gemini-api/docs/file-search)

Index and search across uploaded documents and media files.

Google Maps (/gemini-api/docs/maps-grounding)

Ground responses in real-world geospatial and location data.

Computer use (/gemini-api/docs/computer-use)

Browser automation and screen interaction.

9. Call your own functions

Function calling lets you connect the model to your code. You declare a function's name and parameters, the model decides when to call it and returns structured arguments, and you execute it locally and send the result back.

Stateful (recommended) Stateless (#stateless)
(#stateful-recommended)

Python JavaScript (#javascript) REST (#rest)
(#python)

```
import json
from google import genai

client = genai.Client()

weather_tool = {
    "type": "function",
    "name": "get_current_temperature",
    "description": "Gets the current temperature for a given location.",
    "parameters": {
        "type": "object",
        "properties": {
            "location": {
                "type": "string",
                "description": "The city name, e.g. San Francisco",
            },
        },
        "required": ["location"],
    },
}

available_functions = {
    "get_current_temperature": lambda location: {
        "location": location, "temperature": "22", "unit": "celsius"
    },
}
```

```
user_input = "What is the temperature in London?"
previous_id = None

while True:
    interaction = client.interactions.create(
        model="gemini-3.6-flash",
        input=user_input,
        tools=[weather_tool],
        previous_interaction_id=previous_id,
    )

    function_results = []
    for step in interaction.steps:
        if step.type == "function_call":
            result = available_functions[step.name](**step.arguments)
            print(f"Called {step.name}({step.arguments}) → {result}")
            function_results.append({
                "type": "function_result",
                "name": step.name,
                "call_id": step.id,
                "result": [{"type": "text", "text": json.dumps(result)}],
            })

    if not function_results:
        break

    user_input = function_results
    previous_id = interaction.id

print(interaction.output_text)
```

Response:

During Turn 1, the model returns a response with status `requires_action` and the `function_call` step:

```
{
  "id": "v1_Chd...",
  "status": "requires_action",
  "steps": [
    {
      "type": "function_call",
      "id": "call_abc123",
      "name": "get_current_temperature",
      "arguments": {
        "location": "London"
      }
    }
  ],
  "object": "interaction",
  "model": "gemini-3.6-flash"
}
```

After you run the function locally and submit the result (Turn 2), the final completed interaction returns:

```
{
  "id": "v1_Chd...",
  "status": "completed",
  "steps": [
    {
      "type": "function_call",
      "id": "call_abc123",
      "name": "get_current_temperature",
      "arguments": {
        "location": "London"
      }
    },
    {
      "type": "model_output",
      "content": [
        {
          "type": "text",
          "text": "The temperature in London is currently 22°C."
        }
      ]
    }
  ],
  "object": "interaction",
  "model": "gemini-3.6-flash",
}
```

For advanced features like parallel function calling or function choice modes, see the function calling guide (/gemini-api/docs/function-calling).

10. Run a managed agent

Managed agents run in a remote sandbox with access to tools like code execution and file management. Pass an `agent` instead of a `model` and set `environment="remote"`.

Python JavaScript (#javascript) REST (#rest)
(#python)

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    agent="antigravity-preview-05-2026",
    input="Write a Python script that generates the first 20 Fibonacci numbers",
    environment="remote",
)
print(f"Environment: {interaction.environment_id}")
print(interaction.output_text)
```

You can also define and save custom agents (/gemini-api/docs/custom-agents) with your own instructions, skills, and data sources.

Quickstart (/gemini-api/docs/managed-agents-quickstart)

Make your first agent call, stream responses, and build a custom agent.

Antigravity Agent (/gemini-api/docs/antigravity-agent)

Capabilities, tools, multimodal input, and pricing for the default agent.

Agents in AI Studio (/gemini-api/docs/aistudio-agents)

Visual playground for prototyping agents without writing code.

11. Run tasks in the background

Set `background=True` to run long tasks asynchronously. Poll for results with `interactions.get()`. For more details, see the Background execution guide (/gemini-api/docs/background-execution).

Python (#python) JavaScript (#javascript) REST (#rest)

```
import time
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.6-flash",
    input="Write a detailed analysis of the impact of artificial intelligence o
    background=True,
)
print(f"Started background task: {interaction.id}")
print(f"Status: {interaction.status}")

# Poll for completion
while True:
    result = client.interactions.get(interaction.id)
    print(f"Status: {result.status}")
    if result.status == "completed":
        print(f"\nResult:\n{result.output_text}")
        break
    elif result.status == "failed":
        print(f"Failed: {result.error}")
        break
    time.sleep(5)
```

Response:

The initial response returns immediately with status `in_progress`:

```
{
  "id": "v1_abc123",
  "status": "in_progress",
  "object": "interaction",
  "model": "gemini-3.6-flash"
}
```

Once the background task is fully executed, checking the interaction state returns:

```
{
  "id": "v1_abc123",
  "status": "completed",
  "steps": [
    {
      "type": "model_output",
      "content": [
        {
          "type": "text",
          "text": "Artificial intelligence has transformed modern healthcare in sever
        }
      ]
    }
  ],
  "object": "interaction",
  "model": "gemini-3.6-flash",
}
```

Read about running models and agents asynchronously in the background execution guide (/gemini-api/docs/background-execution).

What's next

- Background execution (/gemini-api/docs/background-execution): Run long-running tasks asynchronously and manage state.
- Text generation (/gemini-api/docs/text-generation): System instructions, generation config, and advanced text patterns.
- Image generation (/gemini-api/docs/image-generation): Aspect ratios, image editing, and style references.
- Image understanding (/gemini-api/docs/image-understanding): Classification, object detection, and visual Q&A.
- Thinking (/gemini-api/docs/thinking): Use chain-of-thought reasoning for complex tasks.
- Function calling (/gemini-api/docs/function-calling): Parallel, compositional, and constrained function modes.
- Google Search (/gemini-api/docs/google-search): Grounding, citations, and search suggestions.

- Managed Agents (/gemini-api/docs/managed-agents-quickstart): Pre-built agents with code execution and file management.
- Deep Research (/gemini-api/docs/deep-research): Autonomous multi-step research with planning and synthesis.
- Structured output (/gemini-api/docs/structured-output): JSON schemas, enums, and recursive type definitions.

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-07-30 UTC.